

プロジェクト概要: AkiNavi HR-AI (アキナビ HR-AI)

1. ビジョン

「案件の空き」と「人材（HR）リソース」をAIで最適に結びつける、高精度・高品質なマッチング基盤。バグゼロを目指した徹底的なテスト、こまめなGit管理、および将来の担当者が即座に再現可能なドキュメント完備をゴールとする。

2. 技術スタック

- **Frontend:** React 19 (Vite 8), TypeScript, Tailwind CSS v4, TanStack Query v5
- **Backend/DB:** Supabase (PostgreSQL, Edge Functions, Realtime, pg_cron, pg_net)
- **AI (ブラウザ) :** Google Gemini デフォルト `gemini-2.5-flash-lite` (`VITE_GEMINI_MODEL` で上書き可)・マルチモーダル対応 (画像解析)
- **ファイルパース (ブラウザ) :** `pdfjs-dist` (PDF)・`xlsx` (Excel)・`mammoth` (Word) — `src/lib/fileParser.ts`
- **AI (サーバー・自動取り込み) :** Cerebras `llama3.1-8b` → Groq `llama-3.1-8b-instant` → Gemini `gemini-2.5-flash-lite` のフォールバック順 — Supabase Edge Function `inbound-email`
- **AI (切替・フロントのみ) :** `VITE_AI_PROVIDER=gemini / openai` — OpenAI は未実装スタブ
- **メール自動取り込み (現行・稼働中) :** Microsoft Graph API ポーリング + Supabase `pg_cron` (Make.com不要・完全無料・5分間隔)
- **メール自動取り込み (旧・現在停止中) :** Make.com → Pipedream (いずれも無料枠超過により運用停止)
- **Testing:** Vitest, React Testing Library, MSW (Mock Service Worker)
- **Deployment:** Vercel (Frontend), Supabase (Backend)

3. 開発・品質管理工程表 (人間とClaude Codeの共同作業)

【Phase 0】 リポジトリ準備・初期化・インフラ開通

1. **【人間】手作業:** GitHubでPrivateリポジトリ作成、Vercelインポート、APIキー取得
2. **【Claude】作業:** Vite + React + TypeScriptプロジェクト初期化、主要ライブラリインストール、Git初期化・push

【Phase 1】 DB基盤と環境構築

1. **【Claude】作業:** jsonbとGINインデックスを用いた汎用DB設計(SQL)の提示
2. **【人間】手作業:** SupabaseでSQL実行、`.env` / Vercel環境変数の設定
3. **【Claude】作業:** 接続確認後、DBスキーマ定義をcommit & push

【Phase 2】 コアロジック開発 & 単体テスト

1. **【Claude】作業:** AI解析エンジン (Edge Functions) およびスコアリングロジックの実装
2. **【Claude】開発:** 単体テスト (Unit Test) の記述
3. **【Claude】作業:** テスト通過を確認し、commit & push

【Phase 3】UI実装 & 結合テスト

- 1. **[Claude] 作業:** ランキング表示、マッチング画面、提案履歴機能の実装
- 2. **[Claude] 開発:** 結合テスト (Integration Test) の記述
- 3. **[人間] 手作業:** 実際のメールデータを用いた解析・マッチング精度の最終検証
- 4. **[Claude] 作業:** 完了後、commit & push

【Phase 4】Make.com 連携・UI改善・デモ環境整備 （完了・メール連携は停止中）

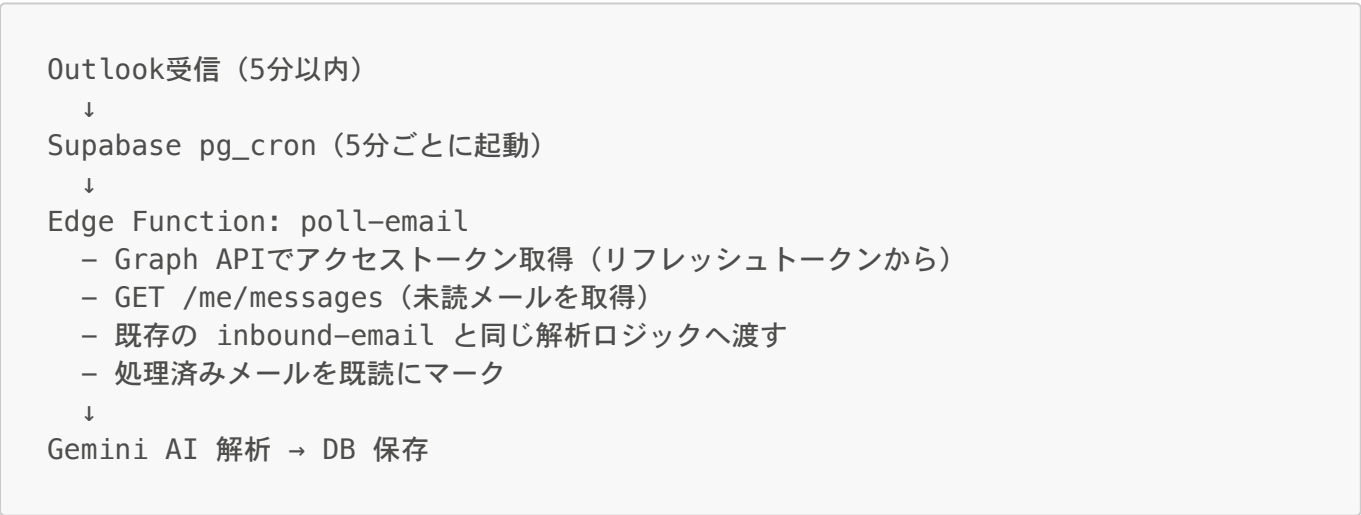
- 1. **[Claude] 作業:** Make.com (Outlook) と連携した自動解析フローの実装
 - フロー: メール受信 → Make.com が検知 → Edge Function `inbound-email` → Gemini AI 解析 → DB 保存
 - 人材用メール: `akinavi.hr.ai.voice.human@outlook.jp`
 - 案件用メール: `akinavi.hr.ai.voice.project@outlook.jp`
- 2. **[Claude] 作業:** AI解析精度の継続的改善（プロンプトチューニング）
- 3. **[Claude] 作業:** Google Drive / Sheets / Docs リンクの自動取得機能実装
- 4. **[Claude] 作業:** デモ環境（data_env）分離・DemoSeedPanel実装
- 5. **[Claude] 作業:** 人材・案件の編集機能・最終更新者/日時表示
- 6. **[人間] 判断:** Make.com・Pipedreamともに無料枠超過 → Microsoft Graph API ポーリングへ移行決定

【Phase 4.5】Microsoft Graph API ポーリングへ移行 （完了・稼働中）

方針

Make.com・Pipedream等の外部SaaSを廃止し、**完全無料・永続稼働**のメール自動取り込みを実現する。

新フロー:



コスト試算（すべて無料枠内）：

コンポーネント	無料枠	予想消費量（5分に1回）
pg_cron	制限なし	約 8,640回/月
pg_net	制限なし	約 8,640回/月
Edge Functions	500,000回/月	約 8,640回/月

コンポーネント	無料枠	予想消費量 (5分に1回)
Microsoft Graph API	無料	約 8,640回/月

作業一覧

【人間】手作業（Claude Codeでは実施不可）

1. Azureアプリ登録（所要: 約30分）

- https://portal.azure.com にアクセス（Microsoftアカウントでログイン・無料）
- 「Microsoft Entra ID」→「アプリの登録」→「新規登録」
- 名前: 任意（例: **akinavi-mail-poller**）
- サポートされるアカウントの種類: 「個人用Microsoftアカウントのみ」を選択
- リダイレクトURL: **http://localhost** を追加
- 登録後、以下を控える:
 - **クライアントID（アプリケーションID）**
 - 「証明書とシークレット」→「新しいクライアントシークレット」→ **クライアントシークレット（値）**
- 「APIのアクセス許可」→「アクセス許可の追加」→ Microsoft Graph → 委任されたアクセス許可
 - **Mail.Read** を追加
 - **Mail.ReadWrite**（既読マーク用）を追加

2. リフレッシュトークン取得（2アカウント分）（所要: 約30分）

- 以下のURLをブラウザで開き、**human@outlook.jp** でログイン:

```
https://login.microsoftonline.com/consumers/oauth2/v2.0/authorize
?client_id=<クライアントID>
&response_type=code
&redirect_uri=http://localhost
&scope=offline_access Mail.Read Mail.ReadWrite
```

- ログイン後にリダイレクトされるURL（**http://localhost/?code=...**）から **code=** の値を控える
- Claude Codeに渡してリフレッシュトークンに交換（次のClaude作業で実施）
- project@outlook.jp** でも同様に繰り返す

3. Supabase Secrets に登録（所要: 約10分）

- Supabase Dashboard → Project Settings → Edge Functions → Secrets
- 以下を追加:

シークレット名	値
GRAPH_CLIENT_ID	AzureのクライアントID
GRAPH_CLIENT_SECRET	Azureのクライアントシークレット

シークレット名	値
GRAPH_REFRESH_TOKEN_HUMAN	human@outlook.jp のリフレッシュトークン
GRAPH_REFRESH_TOKEN_PROJECT	project@outlook.jp のリフレッシュトークン

4. SupabaseでSQL実行（所要: 約5分）

- `supabase/migrations/add_email_polling_cron.sql` をSQL Editorで実行
- `pg_cron`・`pg_net` の有効化とスケジュール登録

【Claude Code】作業（完了）

1. Edge Function `poll-email` の実装

- `supabase/functions/poll-email/index.ts` を新規作成
- 4アカウント（human/prod, project/prod, human/demo, project/demo）を `Promise.allSettled` で並列処理
- リフレッシュトークンを `app_config` テーブルでローテーション保存（フォールバック: Secrets）
- `resolveCallKey(): INBOUND_CALL_KEY → SUPABASE_SERVICE_ROLE_KEY → SUPABASE_ANON_KEY` の順で `eyJ` 始まりJWTを使用
- 処理失敗時は `markAsUnread` で元に戻し次回再試行

2. `pg_cron`マイグレーションSQL作成

- `supabase/migrations/add_email_polling_cron.sql` を作成
- `pg_cron`・`pg_net` の有効化、`app_config` の UNIQUE 制約追加
- 5分ごとに `poll-email` を起動するスケジュール登録

3. 動作確認: 4アカウント全て正常稼働（未読メール → 解析 → DB保存 → 既読マーク）

【Phase 4.6】Microsoft OAuth UI連携機能の追加 （完了）

Supabase Secrets への手動リフレッシュトークン登録を廃止し、設定画面からワンクリックで Microsoft アカウントを連携できる OAuth フローを実装した。

フロー

設定画面で「連携する」をクリック

↓

Edge Function `microsoft-oauth` (step=start) → authorize URL 生成

↓

ブラウザが Microsoft ログインページへリダイレクト

↓

ユーザーがログイン・権限承認

↓

Microsoft が `<origin>/auth/callback?code=...&state=<account>` にリダイレクト

↓

`AuthCallbackPage` が code を受け取り Edge Function `microsoft-oauth` (step=callback) を呼び出す

↓
Edge Function が code → refresh_token を交換し app_config に保存
↓
成功メッセージ表示・設定ページへ誘導

追加・変更ファイル

ファイル	変更内容
supabase/functions/microsoft-oauth/index.ts	新規作成：authorize URL 生成 & code 交換 Edge Function
src/pages/AuthCallbackPage.tsx	新規作成：OAuth コールバック専用ページ
src/pages/SettingsPage.tsx	変更：Microsoft アカウント連携セクション追加
src/App.tsx	変更： <code>/auth/callback</code> パスで AuthCallbackPage を表示
src/lib/db/emailSettings.ts	変更： <code>getConnectionStatuses()</code> 関数追加

app_config キー

キー	内容
graph_rt_human_prod	human@outlook.jp (prod) のリフレッシュトークン
graph_rt_project_prod	project@outlook.jp (prod) のリフレッシュトークン
graph_rt_human_dev	human dev (demo) のリフレッシュトークン
graph_rt_project_dev	project dev (demo) のリフレッシュトークン
graph_connected_<account>	連携済みフラグ (" <code>true</code> " / 未設定)

Azure アプリへの追加設定（手動）

- Azure ポータル → アプリ登録 → 認証 → リダイレクト URI に `<origin>/auth/callback` を追加すること

【Phase 4.7】Box連携 （実装完了・人間手作業待ち）

BoxはOAuth2なしで機械的なファイル取得ができないため、古いWindows PCをデータ取得専用機とした **box-downloader** バッチを別プロジェクトで作成し、Googleスプレッドシートをキューとして本アプリと連携する設計。

全体フロー

- ① メール受信 (inbound-email 改修済み)
 - メール本文中の Box URL (app.box.com/s/xxx) を検出
 - candidates.box_url に保存、box_status = 'pending' で人材を仮登録

- Googleスプレッドシートの boxurl 列に書き込み（キュー登録）
- ② box-downloader バッチ（別プロジェクト・Windows PC・1日1回手動 or タスクスケジューラ）

- スプレッドシートの「実施=空欄」行を読み込む

- Playwright (Chromium) で Box 共有URLへアクセスしファイルをダウンロード

- Google Drive の指定フォルダへアップロード

- スプレッドシートの driveurl 列・実施列（成功/失敗）を更新
- ③ enrich-candidate バッチ（Supabase Edge Function・毎日 JST 3:00 自動実行）

※ box-downloader の実行（②）が完了した後に走るよう時刻設定

- Googleスプレッドシートを読み込む（実施=成功 & driveurl あり）

- candidates テーブルで box_url が一致 & box_status='pending' の人材を検索

- Drive URL からファイルを取得（fetchGoogleLinks の既存ロジックを流用）

- Gemini で再解析 → 既存レコードを UPDATE

- box_status = 'enriched' に更新

スプレッドシート構造（box-downloaderと共有キュー）

列	名前	書き込み元	内容
A	boxurl	inbound-email	Box共有URL
B	driveurl	box-downloader	Google Drive URL（アップロード後）
C	実施	box-downloader	空欄 / 成功 / 失敗

DB変更（candidatesテーブル）

カラム	型	内容
box_url	text	メールから抽出したBox共有URL
box_status	text	pending（未処理） / enriched（更新済み） / failed（失敗）

新規ファイル一覧

ファイル	内容
supabase/migrations/add_box_columns.sql	candidates に box_url, box_status 追加
supabase/functions/enrich-candidate/index.ts	スプレッドシート読み込み → Drive取得 → 再解析 → UPDATE
supabase/migrations/add_enrich_cron.sql	enrich-candidate を毎日 JST 3:00 に起動する pg_cron
../box-downloader/	別プロジェクト (Node.js/TypeScript/Playwright)

改修ファイル一覧

ファイル	変更内容
supabase/functions/inbound-email/index.ts	Box URL 検出 → スプレッドシート書き込み + candidates.box_url 保存

必要な Supabase Secrets

シークレット名	値
GOOGLE_SERVICE_ACCOUNT_JSON	Googleサービスアカウント JSON (1行に圧縮)
BOX_SPREADSHEET_ID	キュー用スプレッドシートのID

【人間】手作業

1. Googleサービスアカウント作成 (所要: 約20分)

- Google Cloud Console → IAM と管理 → サービスアカウント → 新規作成
- Google Sheets API と Google Drive API を有効化
- キー (JSON) をダウンロード → 1行に圧縮して Supabase Secrets の `GOOGLE_SERVICE_ACCOUNT_JSON` に登録
- box-downloader の `auth/service-account.json` にも同じJSONを設置

2. スプレッドシート作成・共有 (所要: 約5分)

- Googleスプレッドシートを新規作成
- 1行目にヘッダー: `boxurl / driveurl / 実施`
- サービスアカウントのメールアドレスを「編集者」として共有
- スプレッドシートのIDを Supabase Secrets の `BOX_SPREADSHEET_ID` に登録

3. SupabaseでSQL実行 (所要: 約5分)

- `supabase/migrations/add_box_columns.sql` をSQL Editorで実行
- `supabase/migrations/add_enrich_cron.sql` の `YOUR_PROJECT_REF` と `YOUR_SERVICE_ROLE_KEY` を書き換えてから実行

4. box-downloaderのセットアップ (所要: 約15分)

- `setup.bat` を実行して依存パッケージをインストール
- `setup-drive-auth.bat` を実行してGoogleドライブOAuth2認証を完了
- `config.json` にスプレッドシートURL・ドライブフォルダIDを設定
- `run.bat` で動作確認

【Phase 5】最終納品ドキュメント作成 (未着手)

- [Claude] 作業:** システム構成図のメンテナンス (README.md に Mermaid 図あり)
- [Claude] 作業:** 操作マニュアルのメンテナンス (`docs/Sales_Manual.md` / `docs/Sales_Manual.pdf`・営業担当者向け)
- [Claude] 完了:** 全成果物を commit & push し、納品完了

4. Claude Codeへの重要な行動指針

- **正の所在:** 仕様・挙動の優先順位は **本リポジトリのソース** と **README.md**。本ファイル (**CLAUDE.md**) はそれに追従するメモであり、食い違いがあれば **ソースを正として本ファイルを更新**すること。
- **こまめな Git 操作:** 機能実装単位、またはテスト通過ごとに、意味のあるメッセージと共に **commit & push** を行うこと。
- **バグゼロの追求:** ロジックには必ずテストコードを付随させ、テスト項目書をエビデンスとして出力すること。
- **ドキュメントの対象読者:**
 - README/構成図は「後任エンジニア」が最短で再現できるように。
 - 操作マニュアルは「非IT営業職」がIT用語なしで理解できるように。

5. データベース構成

`candidate_skills` のカテゴリ CHECK 制約は `supabase/migrations/add_candidate_skills.sql` を正とする。

テーブル一覧

テーブル	用途
<code>candidates</code>	人材マスタ。スキル・経歴・raw_profile を保持。 <code>data_env</code> (<code>prod</code> <code>demo</code>) で論理分離。 <code>box_url</code> / <code>box_status</code> でBox連携状態を管理
<code>projects</code>	案件マスタ。必要スキル・予算・raw_data を保持。 <code>data_env</code> 同上
<code>submissions</code>	マッチング提案履歴。スコア・AI要約を保持。 <code>data_env</code> 同上
<code>candidate_skills</code>	スキルをカテゴリ別に分解して保持（検索最適化・14カテゴリ）
<code>ai_logs</code>	AI解析の実行ログ（モデル・所要時間・結果・エラー）
<code>app_config</code>	アプリ全体設定

candidate_skills の14カテゴリ

カテゴリ	内容
<code>languages</code>	プログラミング言語・クエリ言語
<code>frameworks</code>	フレームワーク
<code>libraries</code>	ライブラリ、UIキット等
<code>os</code>	OS
<code>databases</code>	RDB・NoSQL・KVS
<code>dwh</code>	DWH・BI 等
<code>clouds</code>	クラウドサービス
<code>infrastructures</code>	インフラ技術（コンテナ・IaC 等）

カテゴリ	内容
tools	Git, Jira, Slack, Notion, BIツール等
methodologies	PM・マネジメント系
certifications	資格試験等
design	デザイン・クリエイティブ系
marketing	マーケティング・集客系
others	その他

6. 実装済み機能の詳細

メール自動受信（現行・ポーリング方式）

poll-email と **inbound-email** の役割分担

```
pg_cron（5分ごと）
↓
【poll-email】メール取得係
- Microsoft Graph API で Outlook の未読メールを取得
- AI種別判断（有効時）： candidate / project / other を判定
- 処理済みメールを既読マーク（重複防止）
- メール内容を inbound-email に HTTP POST で渡す（1件ずつ）
↓
【inbound-email】解析・保存係（Make.com時代から存在）
- STEP0-2： メタ情報・本文・添付の受け取りと検証
- STEP3： Word/Excel 添付をテキスト変換
- STEP4： メール本文中の Google Drive リンクを取得
- STEP5： AI（Cerebras → Groq 8b-instant → Gemini フォールバック順）で人材/
案件情報を解析
- STEP6-7： 解析結果を candidates / projects テーブルに DB 保存
```

ポイント: **inbound-email** は今も現役。**poll-email** は「Outlookからメールを取ってきて **inbound-email** に渡す橋渡し役」。Make.com が廃止された後も **inbound-email** の解析ロジックはそのまま流用している。

- **Edge Function:** `supabase/functions/poll-email/index.ts`（Phase 4.5 で実装）
- **スケジューラ:** Supabase pg_cron（5分ごとに起動）
- **認証:** Microsoft Graph API（OAuthリフレッシュトークン方式）
- 人材用: `akinavi.hr.ai.voice.human@outlook.jp`
- 案件用: `akinavi.hr.ai.voice.project@outlook.jp`
- 処理済みメールは既読マークで重複取得を防止

メール設定UI（`src/pages/SettingsPage.tsx`）

- **設定タブ** をナビゲーションに追加（`src/components/Layout.tsx`）

- **メールアドレス設定:** 人材用・案件用アドレスを表示用に `app_config` へ保存（参照用。実認証情報は Supabase Secrets）
- **AI種別判断:** 同じ受信箱に人材・案件メールが混在する場合、Gemini AI で `candidate / project / other` を自動分類
 - 有効時は人材用アカウントのみポーリング（同一受信箱を2重処理しない）
 - `other` と判断されたメールは既読マークしてスキップ
 - 10秒タイムアウト、失敗時は `candidate` にフォールバック
- **全件取り込みセクション:** 削除済み（7日以上前のデータは不要のため）
- **DB:** `src/lib/db/emailSettings.ts` で `app_config` から設定を読み書き
- **マイグレーション:** `supabase/migrations/add_email_settings.sql`

全件取り込みモード（`poll-email` Edge Function）

- **通常モード（incremental）:** 未読メールのみ取得（通常運用）
- **全件モード（full）:** `email_full_import_since` 以降の全メールを順次取得（isRead フィルターなし）
 - 1バッチ最大20件、`@odata.nextLink` でページネーション継続
 - バッチごとに `nextLink` を `app_config` (`email_full_import_nextlink_<configKey>`) に保存し、5分ごとに続きから再開
 - 全アカウント完了時に自動で incremental モードに戻る
- **UI:** 設定ページの全件取り込みセクションは**削除済み**（7日以上前のデータは不要のため運用上使用しない）
 - 必要の場合は Supabase SQL Editor で `app_config` の `email_poll_mode` を `full`、`email_full_import_since` を開始日付に手動設定すること

メール自動受信（旧方式・停止中）

- **Make.com:** 無料枠 約1,000ops/月 → 超過により停止
- **Pipedream:** 無料枠 10クレジット/日 → 当日中に超過・停止
- **Power Automate:** 外部URLへのHTTP POSTがプレミアムコネクタのため不採用

Edge Function `inbound-email`（`supabase/functions/inbound-email/index.ts` 準拠）

- **データ環境:** ボディまたはクエリの `mode / data_env` (`prod | demo | dev`)。省略時は `prod`
- **Secrets:** `GEMINI_API_KEY`、`SUPABASE_URL`、`SUPABASE_SERVICE_ROLE_KEY`
- **`GEMINI_INBOUND_TIMEOUT_MS`:** 1回のGemini待ち上限（未設定時 38,000ms）
- **`INBOUND_MAKE_SOFT_FAIL`:** 例外時も HTTP 200 + `ok:false` で返す（Make/外部サービス停止回避）
- 本文・添付とも空: HTTP 200 + `skipped`

論理データ環境 `data_env`

- `prod / demo` を同一Supabase内で分離（`data_env` カラムでフィルター）
- **デモ解除:** `VITE_DEMO_KEY` と URL クエリ `?demo=<鍵>` でトグル
- 解除時にヘッダの「データ」セレクトが表示される
- デモ UI 未解除時は常に `prod` 固定

マッチング画面

マッチング方式の比較と使い分け

	高速モード (fast)	全件モード (full)	自動バッチ (daily cron)
実行タイミング	手動 (UIボタン)	手動 (UIボタン)	毎朝9時 自動
案件→人材	最大20名 (スキルスコア 上位)	全候補者	スキル重複フィルター後 最大40 名
人材→案件	最大10件 (スキルスコア 上位)	全案件	未対応
Gemini呼び出し数	少 (数〜数十回)	多 (候補者×案件 数)	中 (スキル一致者のみ)
速度	速い (数秒〜数十秒)	遅い (数分〜)	気にしない (バックグラウンド)
操作	必要	必要	不要
追加実装	不要 (既存)	不要 (既存)	auto-match Edge Function + pg_cron

推奨する使い分け:

- 日常運用 → 自動バッチに任せる (毎朝9時に前日登録分が自動でマッチング済み)
- 急ぎで確認したい → 手動・高速モード (数秒〜数十秒で完了)
- 念入りにやりたい → 手動・全件モード (時間がかかるが全候補と照合)

自動バッチ (supabase/functions/auto-match/index.ts)

- スケジュール: 毎日 0:00 UTC (日本時間 9:00)
- 対象: 直近25時間以内に登録された prod 案件
- 除外: 既に submissions が存在するペア、accepted ステータスの人材
- マイグレーション: supabase/migrations/add_auto_match_cron.sql
 - YOUR_PROJECT_REF と YOUR_SERVICE_ROLE_KEY を書き換えてから Supabase SQL Editor で実行すること
- 実行モード: fast (高速・上限あり) / full (全件)
- 高速モード上限: 案件あたり候補 最大20名、人材あたり案件 最大10件
- 一括マッチングのキャンセル: bulkCancelRequestedRef による途中停止対応
- MatchingPage は常時マウント: タブ切替で mutation が中断されないよう hidden で切替

デモシード (src/components/DemoSeedPanel.tsx)

- デモ環境向けに人材・案件のサンプルペアをDB投入

画面構成

- タブは3つに整理（**マッチング結果** / **人材登録** / **案件登録**）
- **提案履歴** / **重複管理** / **解析監視** は実装済みだがナビから非表示

Google Drive / Sheets / Docs 自動取得

- メール本文中のリンクを自動検出・取得
- Sheets → CSV、Docs → txt、Drive PDF → base64化してGeminiに渡す
- 認証不要（リンクを知っている全員が閲覧可の共有設定前提）

ファイルアップロード解析（ブラウザ）

- **実装**: `src/lib/fileParser.ts`
- **PDF（テキストベース）**: `pdfjs-dist` でテキスト抽出 → テキストエリアへ自動転記 → 既存の解析フローへ
- **PDF（スキャン・画像化）**: テキスト抽出不可。画像として添付するか手動テキスト入力が必要
- **Excel（.xlsx/.xls）**: `xlsx`（SheetJS）で全シートをCSV変換 → テキストエリアへ自動転記
- **Word（.docx）**: `mammoth` で本文テキスト抽出 → テキストエリアへ自動転記
- **画像（JPG/PNG等）**: base64変換 → `AnalyzeCandidateRequest.imageFiles` / `AnalyzeProjectRequest.imageFiles` に格納 → Gemini multimodal API（`inlineData`）で解析
- 対応ページ: 人材登録（`CandidatePage.tsx`）・案件登録（`ProjectPage.tsx`）
- 複数ファイル同時選択可。テキスト貼り付けとの併用も可能

AI プロバイダー

- **ブラウザ**: Gemini（既定 `gemini-2.0-flash`）・マルチモーダル対応（テキスト＋画像の同時解析）
- **サーバー（Edge Function）**: Gemini `gemini-2.5-flash` 固定
- フロントは `AIProvider` インターフェースで抽象化（OpenAI切替はスタブのみ）
- `geminiProvider.ts` の `generate()` は `imageFiles` オプション引数でマルチモーダル対応

認証・ニックネーム制

- ログイン機能なし
- 初回アクセス時にニックネームを入力させ `localStorage` に保存

データ重複管理

- email が同じ場合は自動で既存レコードを UPDATE（上書き更新）
- AI が重複疑いと判断した場合は `duplicate_flag = true` を立てるだけ（自動マージ不可）

AI解析ログ（ai_logs）

- 全AI解析呼び出しをDBに記録（モデル名・所要時間ms・結果JSON・エラー）
- 成功・失敗どちらもログに残す